

Maths Extended Essay

When building a binary heap, why is Floyd's algorithm faster than Williams's algorithm?

4000 words

Contents

1	Introduction	2
2	Methodology	2
2.1	The OPS function	2
2.2	O -notation	3
2.2.1	Example scenario	3
2.2.2	Multiplicative change	6
2.2.3	The omission of the base of logarithms	8
2.3	Use of O -notation in this essay	9
3	The priority queue	10
3.1	Binary heap representation of priority queues	12
3.2	Advantage of binary heaps	15
4	Williams's binary heap creation method	18
4.1	Obtaining the OPS function	23
4.2	Finding the upper bound	25
4.3	Finding the lower bound	27
4.4	Obtaining the OOG	28
5	Floyd's binary heap creation method	28
5.1	Obtaining the OPS function	34
5.2	Use of geometry to transform the problem	37
5.3	Obtaining the OOG	39
6	Conclusion	40
7	Bibliography	42

1 Introduction

Priority queues are essential when creating applications that require a weightage to gauge the importance of data within a queue, such as a hospital triage application where patients with the worst injuries are given a greater priority over other patients (Cornell University). While exploring how computers implemented priority queues into applications, I discovered the concept of binary heaps and found two methods of creating a binary heap: R. W. Floyd's heap creation method and J. W. J. Williams's heap creation method (Li et al. 58). Williams is the inventor of the binary heap, introducing his heap creation method alongside the binary heap data structure (Williams). Later, Floyd developed a faster method of creating a binary heap which has been proven to be faster than Williams's method through the use of calculus (Skiena 28). To answer the question: "When building a binary heap, why is Floyd's algorithm faster than Williams's algorithm?", this essay describes and compares the speeds of both algorithms using mathematics.

2 Methodology

To keep explanations as simple as possible, I have decided to use geometry to examine the OPS function (defined in the following subsection) of Floyd's method as well as graphing to understand how Williams's heap creation method works, and why it is typically slower.

2.1 The OPS function

The speeds of algorithms can be expressed as a function which maps the **non-negative integer** N , the input size (the amount of data that needs to be processed) of the algorithm, to the maximum number of operations required for said algorithm

to complete (Eldridge). This function will be referred to as the **OPS function** of an algorithm within this essay. The OPS function of an algorithm is critical as it largely determines the time taken for a procedure, through said algorithm, to be completed since computers take a rather constant amount of time to process each operation (Sirois).

To compare different OPS functions, I will use the O -notation, introduced in the next section.

2.2 O -notation

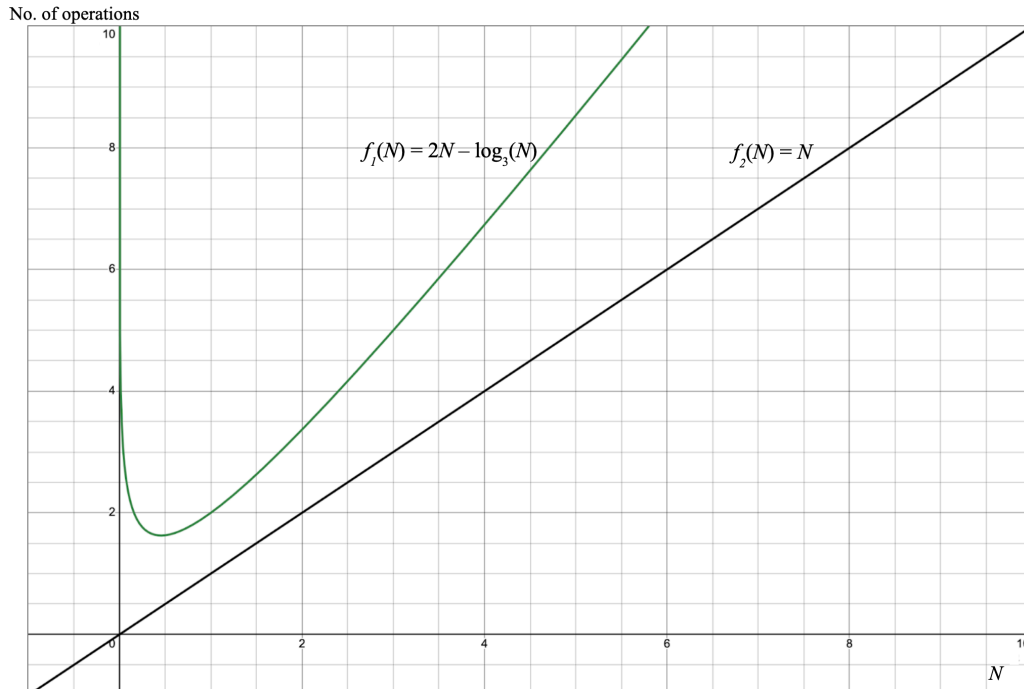
The O -notation is a set of functions, utilised in computer science to describe the speed of an algorithm for a large input size of N (Schwarz; Cormen et al. 43). In other words, it describes the **overall growth** of the OPS function of an algorithm, as N increases. The overall growth of the function is known as the **order of growth (OOG)** of said function (Cormen et al. 44).

This is done by establishing an asymptotic upper bound of a given function **to within a constant factor** (Cormen et al. 44,47). So, for a given function $g(N)$, $O(g(N))$ (the O of $g(N)$) is a set of functions that is greater than or equal to 0 and less than or equal to $k \times g(N)$ where k is some constant, for large values of N (Cormen et al. 47). These characteristics are demonstrated in the following subsection.

2.2.1 Example scenario

For this section, let $f_1(N)$ and $f_2(N)$ represent the OPS function of two separate algorithms where $f_1(N) = 2N - \log_3(N)$ and $f_2(N) = N$. In the graph below, $f_1(N)$ and $f_2(N)$ are drawn.

Figure 1: Example functions for t given small values of N

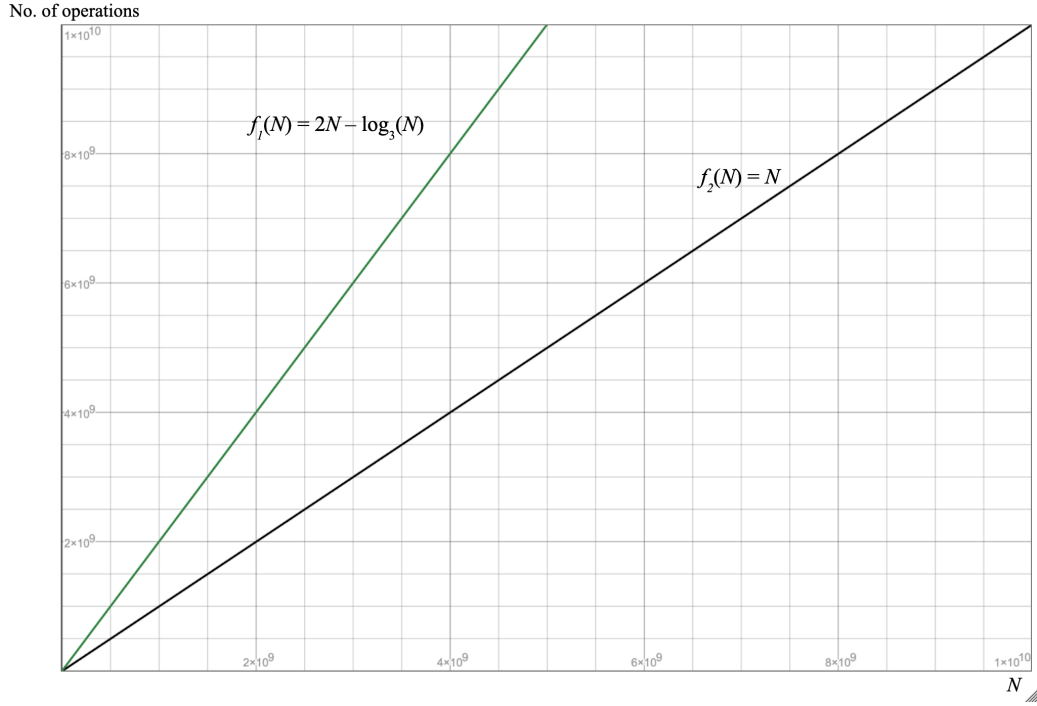


Shown in the figure above, $f_1(N)$ is clearly non-linear while $f_2(N)$ is, where:

$$D_f = \{N \in \mathbb{R}^+ \mid 0 \leq N \leq 10\}$$

The appearance of the functions seems to change for large values of N , where both visually appear linear, as demonstrated below.

Figure 2: Example functions for t given large values of N



In figure 2, the functions $f_1(N)$ and $f_2(N)$ are drawn, where:

$$D_f = \{N \in \mathbb{R}^+ \mid 0 \leq N \leq 10^{10}\}$$

which simulates large values of N , where both functions resemble a linear shape.

Partially explained through the use of limits, when N approaches infinity in $f_1(N)$, $2N \gg \log_3(N)$ and $\log_3(N)$ becomes insignificant when compared to $2N$, where $2N$ is the term with the highest OOG:

$$f_1(N) = 2N - \log_3(N)$$

$$\lim_{N \rightarrow +\infty} \frac{f_1(N)}{N} = 2$$

According to the definition of the O -notation, since there does exist some constant k where $kN \geq f_1(N)$, such as when $k = 2$, for all large values of N , $f_1(N)$ has a

linear OOG, which is $O(N)$. This also applies to $f_2(N)$, which has a linear OOG; the coefficient is a constant, 1. Hence, the OOG of $f_1(N)$ is the same as the OOG of $f_2(N)$:

$$f_1(N) \in O(N)$$

$$f_2(N) \in O(N)$$

Although $f_1(N)$ and $f_2(N)$ have different coefficients, they are said to have the same OOG because they have the same approximate **multiplicative change** as the input size N changes. This property of the O -notation is examined below.

2.2.2 Multiplicative change

Consider the example in which $a(N) = 7N^2$ and $b(N) = 2N^2$, where both are OPS functions of separate algorithms. Although $a(N)$ and $b(N)$ have different coefficients of N^2 , they are both elements of $O(N^2)$ as both algorithms have, approximately, the same multiplicative change when N is large.

In a scenario where $\beta = 2N$ (input size N is doubled), the multiplicative change between $a(N)$ and $a(\beta)$ would be such that $a(N) \times \text{multiplicative change} = a(\beta)$. Likewise, the same multiplicative change between $b(N)$ and $b(\beta)$ would be such that $b(N) \times \text{multiplicative change} = b(\beta)$ since $a(N)$ and $b(N)$ are both elements of $O(N^2)$.

This specific scenario can be proven by first finding the multiplicative change, μ , between $a(N)$ and $a(\beta)$:

$$a(N) = 7N^2$$

$$a(\beta) = a(2N) = 7(2N)^2$$

$$a(N) \times \mu = a(\beta)$$

$$(7N^2) \times \mu = 7(2N)^2$$

$$7\mu N^2 = 28N^2$$

$$\mu = \frac{28}{7}$$

$$= 4$$

I get the same value of μ between $b(N)$ and $b(\beta)$:

$$b(N) = 2N^2$$

$$b(\beta) = b(2N) = 2(2N)^2$$

$$b(N) \times \mu = b(\beta)$$

$$(2N^2) \times \mu = 2(2N)^2$$

$$2\mu N^2 = 8N^2$$

$$\mu = \frac{8}{2}$$

$$= 4$$

In fact, it can be proven that any OPS function that is an element of $O(N^2)$ would have a multiplicative change of 4 when the input of the function, N , is doubled. Let said function be expressed as:

$$\gamma(N) = kN^2 + \text{terms with an OOG lower than quadratic,}$$

where N is large and $k \in \mathbb{R}^+$

And so,

$$\gamma(N) \times \mu \approx \gamma(2N)$$

$$(kN)^2 \times \mu = (k \times 2N)^2$$

$$\mu k^2 N^2 = 4k^2 N^2$$

$$\therefore \mu = 4$$

It is concluded that all elements of $O(N^2)$ would have approximately the same multiplicative change of 4, where N is already large, when its input size is doubled.

Similar to how constant coefficients of N do not affect functions' OOGs, in the context of the O -notation, the **base** of a logarithm also does not affect its OOG.

2.2.3 The omission of the base of logarithms

As you will see in figure 3 within section 2.3, the bases of the logarithms in $O(\log(N))$ and $O(N\log(N))$ are omitted. While it implies the logarithm has a base of 2 (due to the binary system in computer science), it can be shown that $\log_k(N) \in O(\log(N))$, where k is a constant within the set of positive real numbers and $k \neq 1$.

According to the International Baccalaureate, $\log_a(x) = \frac{\log_b(x)}{\log_b(a)}$ where $a^x = b$ if and

only if $x = \log_a(b)$, where $a > 0$ and $b > 0$ and $a \neq 1$. Hence:

$$\begin{aligned}\log_k(N) &= \frac{\log(N)}{\log(k)} \\ &= \frac{1}{\log(k)} \times \log(N) \\ &\in O(\log(N)) \quad \because \frac{1}{\log(k)} \text{ is a constant}\end{aligned}$$

Because $\log_k(N)$ is $\log(N)$ multiplied by a constant, the constant base of a logarithm in the context of the OOG is irrelevant.

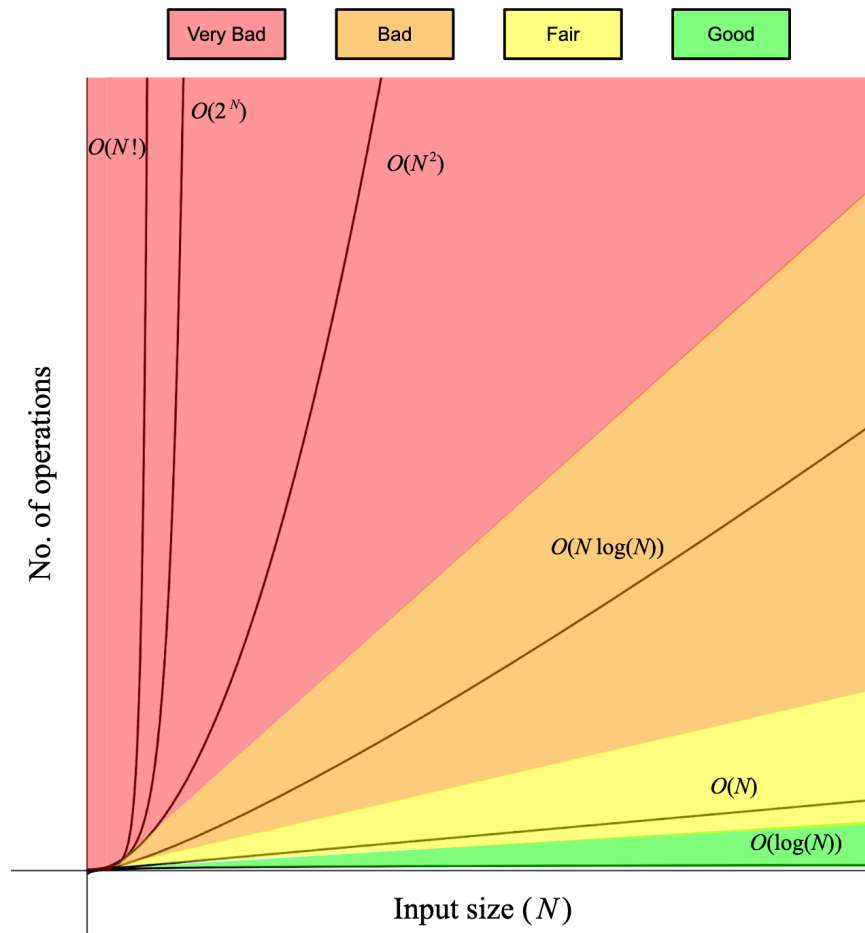
Note that this holds true for elements of $O(N \log(N))$, where:

$$N \log_k(N) = \frac{1}{\log(k)} N \log(N) \in O(N \log(N))$$

2.3 Use of O -notation in this essay

A computer algorithm would be considered as faster if its OPS function, given large inputs, had a smaller OOG, illustrated in the diagram below (Huang).

Figure 3: Classifications of different orders of growth



Interpreted from: <https://www.freecodecamp.org/news/big-o-notation-why-it-matters-and-why-it-doesnt-1674cfa8a23c/>

The different OOGs will be used to describe and compare the speeds of algorithms, such as algorithms that create binary heap, as seen in but not limited to sections 4.4, 5.3 and 6.

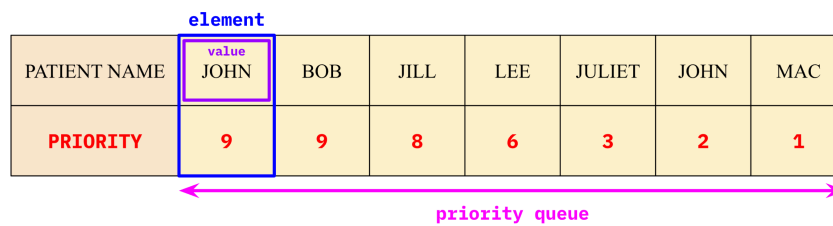
Next, the role of binary heaps in relation to priority queues is discussed.

3 The priority queue

Priority queues are made up of elements such as numbers or characters that have differing levels of priority, usually represented numerically (Cornell University). Pri-

riority queues can exist as a sorted or unsorted sequence of elements (Cornell University). The diagram below illustrates an example of a priority queue, used to triage patients, where the value of an element is the patient's name and their urgency for medical attention is a range of integers, sorted in descending order.

Figure 4: Priority queue (sorted) of an example triage system



PATIENT NAME	value JOHN	BOB	JILL	LEE	JULIET	JOHN	MAC
PRIORITY	9	9	8	6	3	2	1

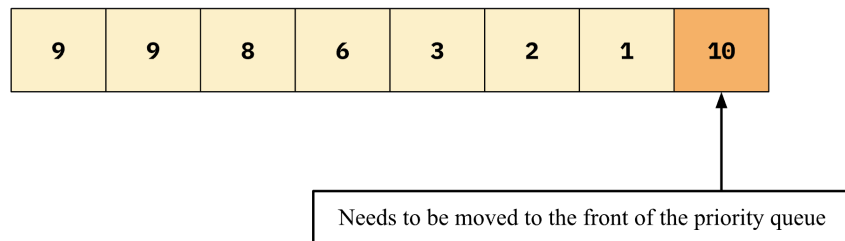
priority queue

The value the elements contain do not affect the order of the priority queue; it is only ordered by the element's priority. It will therefore be omitted from future illustrations.

In a priority queue that is implemented through a sequence of sorted elements, the computer procedure of retrieving the highest and lowest priority elements is the fastest as they are located on each end of the queue while the process of inserting new elements into the queue may require the queue to be rearranged to remain sorted.

In figure 5, an element of priority 10 was added to the back of the priority queue, which was made up of a sorted sequence of elements, in descending order.

Figure 5: Example of a priority queue of sorted elements that needs to be rearranged



Since the element which was just added had a higher priority than all other elements that were in the queue, it has to be shifted to the front of the priority queue, affecting the position of all elements in the queue. Because of this, adding an element would be slower than the retrieval of the highest and lowest priority elements.

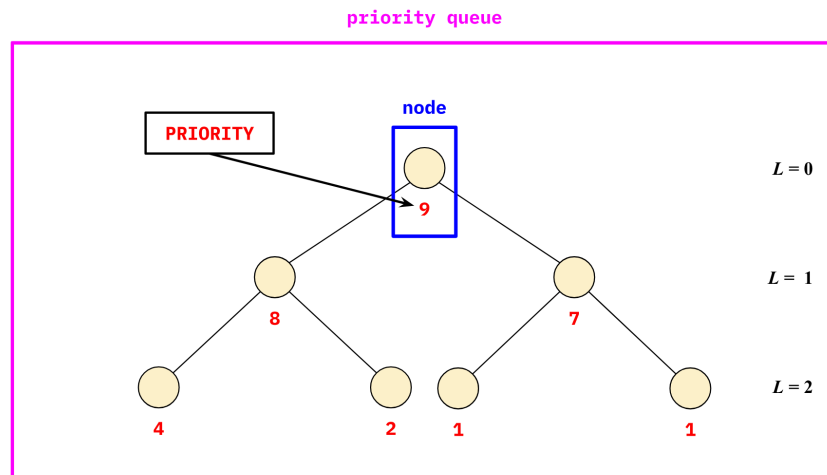
Priority queues can also be implemented using a sequence of unsorted elements (Cornell University). In a sequence of unsorted elements, it is faster to add new elements to the existing priority queue as the elements' order does not matter while it is slower to retrieve the element with the highest priority as a search for the element is required since it may not be found at either end of the queue.

While a priority queue can be represented by a sequence of elements as seen in figure 4, it can also be expressed in the form of a tree called a binary heap.

3.1 Binary heap representation of priority queues

In a binary heap, elements are represented as nodes. Figure 6 illustrates a max binary heap, where the top most node holds the highest priority (Demaine et al.). On the other hand, a min heap would have the top most node holding the lowest priority (National University of Singapore Computing).

Figure 6: Binary heap implementation of a priority queue



★ Note:  is a child node of  and  is a parent node of 

This parent-child behaviour applies to all other nodes and is not limited to the example directly above

In figure 6, the nodes are split into different levels, starting from 0. All binary heaps would have 2^L nodes at level L , for every level, except its lowest level which would have between 1 to 2^L nodes, inclusive (Cornell University). For ease of explanations, only max binary heaps that have 2^L nodes in their lowest level will be used throughout this investigation. Nodes connected to another node with a higher priority are called **child nodes**, with the higher priority nodes being called **parent nodes** (Cornell University). The node at the top of the binary heap is called the **root node** (Cornell University). In order for a binary heap to be valid, all parent nodes in a max heap must not have a lower priority than both its child nodes (Cornell University).

The maximum level a heap has is referred to as its height, H . The relationship between the total number of nodes, N , and H is shown in the table below so that it can be utilised in later calculations.

Table 1: Relationship of N and H

H	0	1	2	3	4	...	h
N	1	3	7	15	31	...	N_h

Though table 1 does not provide a sequence of numbers that is arithmetic or geometric, I realised that N had values that were close to the powers of 2. By adding 1 to all values of N , we can now view table 2 as a geometric sequence with a common ratio of 2.

Table 2: Relationship of $N + 1$ and H

H	0	1	2	3	4	...	h
$N + 1$	2	4	8	16	32	...	$N_h + 1$

With the use of $u_n = u_1 r^{n-1}$, which is the formula to find the n th term of a geometric sequence where u_1 is the first term of the sequence and r is the common ratio, I can now express H in terms of N as shown in (2) and vice versa in equation 1 (International Baccalaureate).

$$u_n = u_1 r^{n-1}$$

$$N + 1 = 2 \times 2^H$$

$$N + 1 = 2^{H+1}$$

$$N = 2^{H+1} - 1 \tag{1}$$

$$H + 1 = \log_2(N + 1)$$

$$H = \log_2(N + 1) - 1 \tag{2}$$

Equations 1 and 2 will be used in future calculations, such as to find the OPS functions of Williams's method and Floyd's method.

This essay focuses on priority queues as binary heaps instead of priority queues as sequences of numbers as it generally takes a smaller number of operations to complete for a combination of operations, discussed in the next section.

3.2 Advantage of binary heaps

Table 3 contains the OOG of the OPS functions for the insertion or removal of the element with the highest priority from a priority queue of N elements that has been implemented through a binary heap, a sorted sequence of elements and a completely unsorted sequence of elements.

Table 3: OOGs of OPS functions for insertion and removal operations

	Binary heap	Sorted sequence of elements	Unsorted sequence of elements
OOG of the OPS function to insert a new element	$O(\log(N))$	$O(N)$	$O(1)$
OOG of the OPS function to remove the highest priority element	$O(\log(N))$	$O(1)$	$O(N)$

Values in the "Binary heaps" column are according to Demaine et al. and the National University of Singapore Computing

Recalling the explanations of a sorted sequence of elements and an unsorted sequence of elements from section 3 and with reference to table 3, an OOG of $O(N)$ indicates that a search involving multiple operations is required while an OOG of $O(1)$, indicates a constant number of operations is required.

The nature of the different values within table 3 is examined with figures 7 and 8, where an element with priority 9 is added to a sorted sequence of elements and a

binary heap, both with 7 existing elements in it.

Figure 7: Illustration of insertion procedure in a sorted sequence of elements

7 swaps are needed to place the new element in its correct position

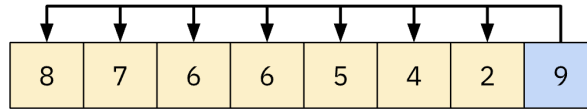
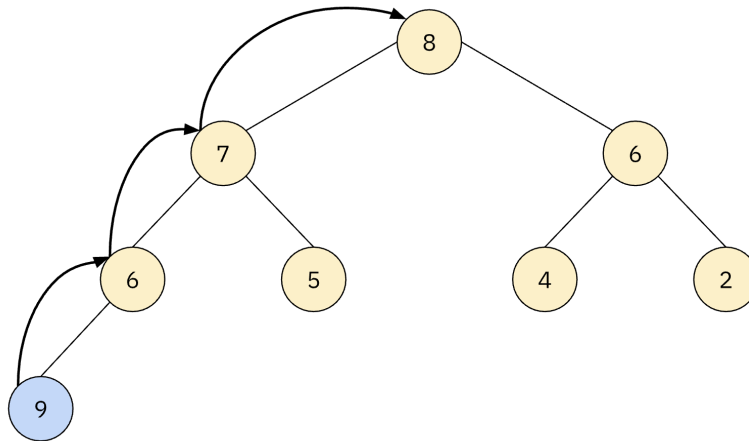


Figure 8: Illustration of insertion procedure in a binary heap

Only 3 swaps are required to place the new element in its correct position



In figure 7, it took 7 swaps to place the newly-added 8th element in its correct position while the binary heap in figure 8 only took 3 swaps. With each swap considered as an operation, the binary heap required less operations for insertion when compared to a sorted sequence of elements.

Combining the values of the insertion and removal procedures from table 3,

OOG of combined procedures in binary heaps

$$\equiv O(\log N) + O(\log N)$$

$$\equiv 2 \times O(\log N)$$

$$\equiv O(\log N)$$

OOG of combined procedures in sequence of numbers

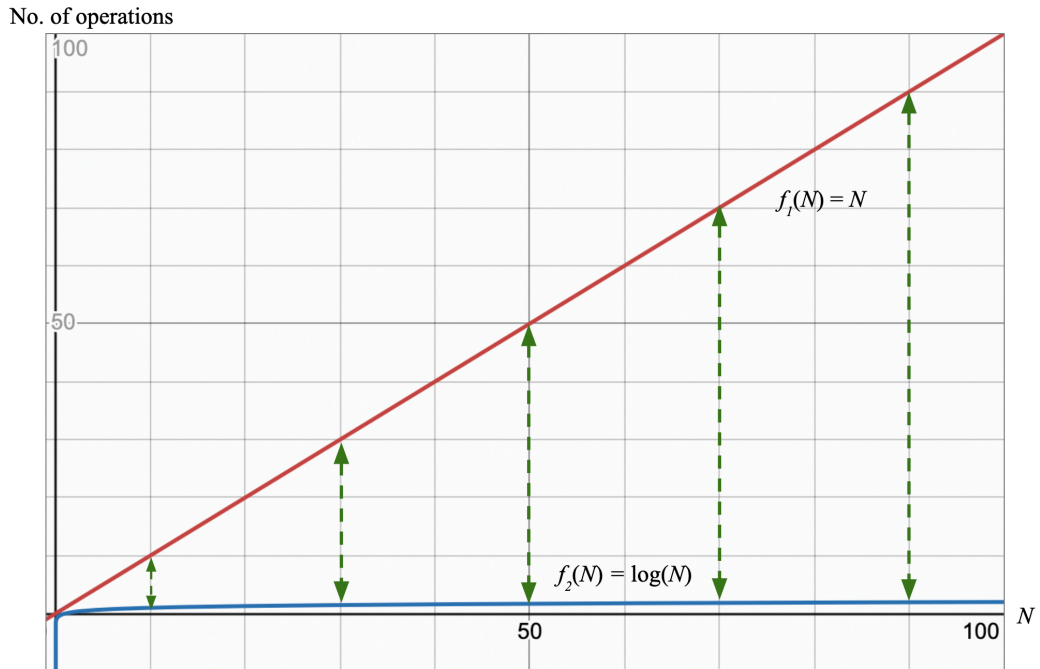
$$\equiv O(1) + O(N)$$

$$\equiv k + O(N), \quad k \in \mathbb{R}^+$$

$$\equiv O(N)$$

For the same mix of operations and input size, the OOG of the combined procedures in binary heaps is generally less than the OOG of the combined procedures in a sequence of numbers, whether unsorted or sorted. This difference grows as N increases, shown in the figure below.

Figure 9: Growth in difference between $O(N)$ and $O(\log N)$



Note that, in figure 9, $f_1(N) \in O(N)$ and $f_2(N) \in O(\log N)$, with the green dotted lines representing the rough difference between the output of $f_1(N)$ and $f_2(N)$ for differ-

ent values of N .

While binary heaps are shown to provide faster procedures in a priority queue, the procedures of creating a binary heap were also improved (Li et al. 58).

4 Williams's binary heap creation method

Initially, binary heaps were created through the use of J.W.J. Williams's method by inserting each element one after the other (National University of Singapore Computing 4; Li et al. 58). Binary heaps are always filled from left to right, only creating a new level when the lowest level is full; the number of nodes at the lowest level is exactly 2^L , for the purpose of this essay (Cornell University).

An example of how a binary heap is created through the use of Williams's method is demonstrated in figure 10. Numbers that have not been added to the binary heap are marked with a yellow background while numbers that have already been added to the binary heap are marked with a green background.

Figure 10: Creation of a binary heap through Williams's method

STEP 1

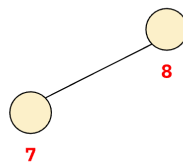
Elements to add:	8	7	6	5	10	4	11
------------------	---	---	---	---	----	---	----



The first element / node is added

STEP 2

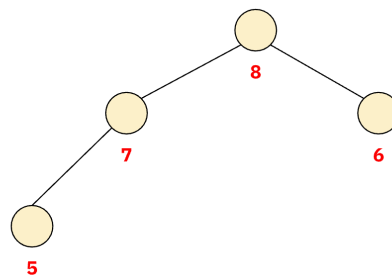
Elements to add:	8	7	6	5	10	4	11
------------------	---	---	---	---	----	---	----



The second node is added

STEP 3 & 4

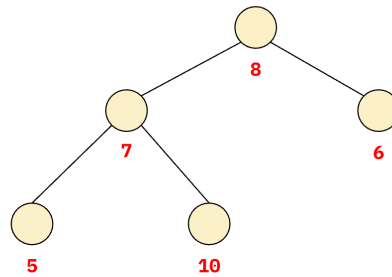
Elements to add:	8	7	6	5	10	4	11
------------------	---	---	---	---	----	---	----



The third and fourth node is added

STEP 5

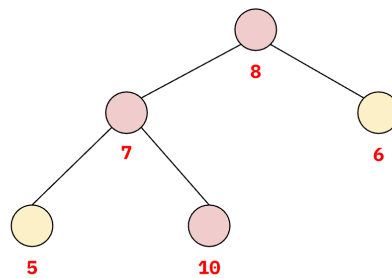
Elements to add:	8	7	6	5	10	4	11
------------------	---	---	---	---	----	---	----



The fifth node is added, but it is in an invalid position.

STEP 6

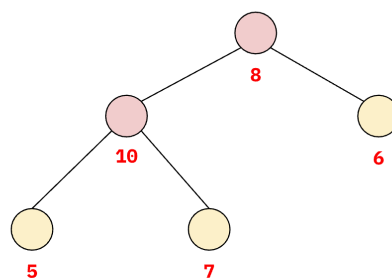
Elements to add:	8	7	6	5	10	4	11
------------------	---	---	---	---	----	---	----



Nodes in invalid positions are identified and are marked in red. $10 \geq 7$ and $10 \geq 8$.

STEP 7

Elements to add:	8	7	6	5	10	4	11
------------------	---	---	---	---	----	---	----



Nodes with priority 10 and 7 are swapped

STEP 8

Elements to add:

8

7

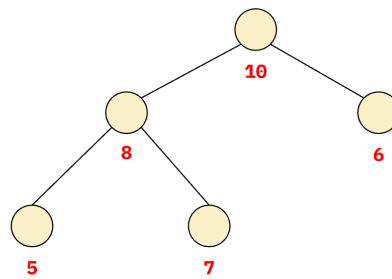
6

5

10

4

11



Nodes with priority 10 and 8 are swapped

STEP 9

Elements to add:

8

7

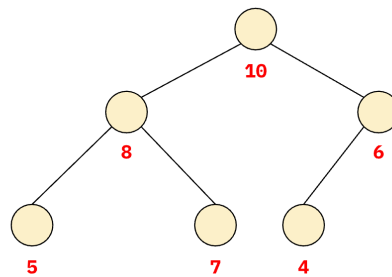
6

5

10

4

11



The sixth node is added

STEP 10

Elements to add:

8

7

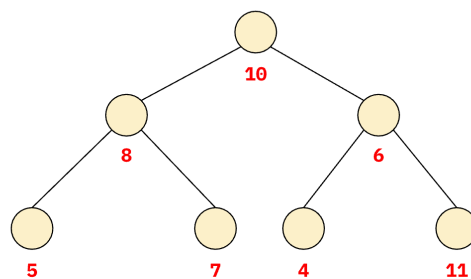
6

5

10

4

11



The last node is added, and it is in an invalid position.

STEP 11

Elements to add:

8

7

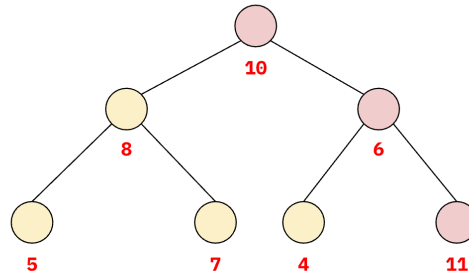
6

5

10

4

11



Nodes in invalid positions are identified and are marked in red. $11 \geq 6$ and $11 \geq 10$.

STEP 12

Elements to add:

8

7

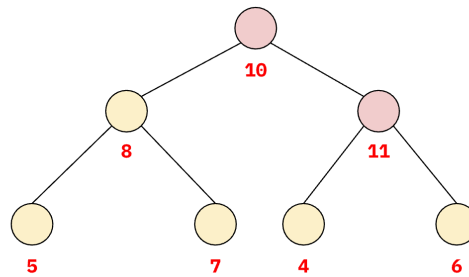
6

5

10

4

11



Nodes with priorities 11 and 6 are swapped

STEP 13

Elements to add:

8

7

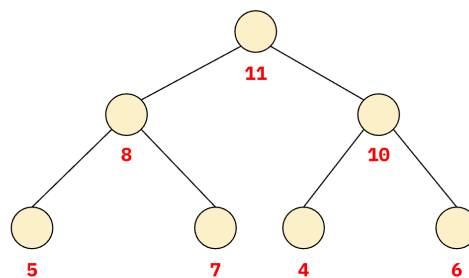
6

5

10

4

11



Nodes with priorities 11 and 10 are swapped

In figure 10, each element that was added had a priority that was lower than the one before it except in steps 5 and 10. This was to demonstrate how Williams's heap creation method swapped nodes that were added in an invalid position and how it did not swap the other nodes that were placed in valid positions.

To describe the speed of Williams's method, I will assume a worst case scenario, where every new element added to the priority queue would be larger than all existing elements in the queue, requiring the rearrangement of nodes all the way up to the root node, as seen with the nodes with priorities 10 and 11 in figure 10. Therefore, as the number of elements increases in a binary heap, the more swaps are needed to create a binary heap. This is explained in the following section.

4.1 Obtaining the OPS function

Let $P(H)$ be the number of operations Williams's heap creation method requires given height H . To find the OOG of the OPS function of Williams's method, I would first need to find $O(P(H))$ and simplify it to be in terms of N .

In table 4, the total number of operations at level L in a binary heap as well as the total number of operations for a binary heap of height H is shown.

Table 4: Table of total no. of operations compared to number of nodes

L	0	1	2	3	4	5	6	...	H
No. of nodes at level L	2^0	2^1	2^2	2^3	2^4	2^5	2^6	...	2^H
No. of operations at level L	0	2	8	24	64	160	384	...	$2^H \times H$
Total no. of operations for a binary heap of height L	0	2	10	34	98	258	642	...	$P(H)$

I discovered that the total number of operations for a binary heap of **height** H is an accumulation of the values of the total number of operations at **levels** less than or equal to H in a binary heap. So, I was able to express $P(H)$ as a series, one term for each level.

$$P(0) = 2^0 \times 0 = 0$$

$$P(1) = (2^0 \times 0) + (2^1 \times 1) = 2$$

$$P(2) = (2^0 \times 0) + (2^1 \times 1) + (2^2 \times 2) = 10$$

$$P(3) = (2^0 \times 0) + (2^1 \times 1) + (2^2 \times 2) + (2^3 \times 3) = 34$$

$\vdots$

$$P(H) = \sum_{i=0}^H (2^i \times i)$$

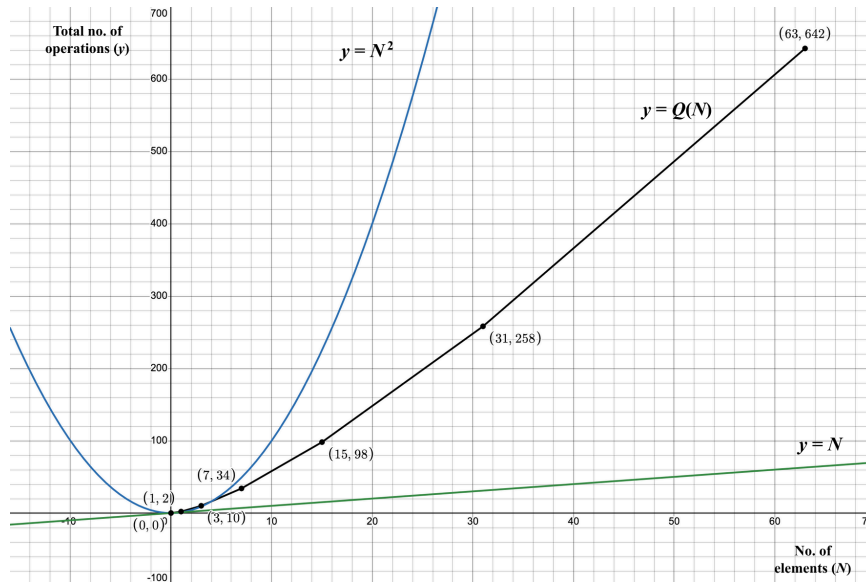
This can also be expressed recursively as

$$P(H) = \begin{cases} 0 & \text{if } H = 0 \\ P(H-1) + 2^H \times H & \text{if } H \geq 1 \end{cases}$$

Unfortunately, simplifying $P(H) = \sum_{i=0}^H (2^i \times i)$ with just the sigma notation is difficult as the series is neither arithmetic nor geometric. Therefore, I decided to plot the relationship of N to the total number of operations for binary heaps with heights 0 to 6.

Let $Q(N) \equiv P(H)$ be the equivalent function that provides the OPS function of Williams's method, in a binary heap.

Figure 11: Graphed relationship between N and $Q(N)$



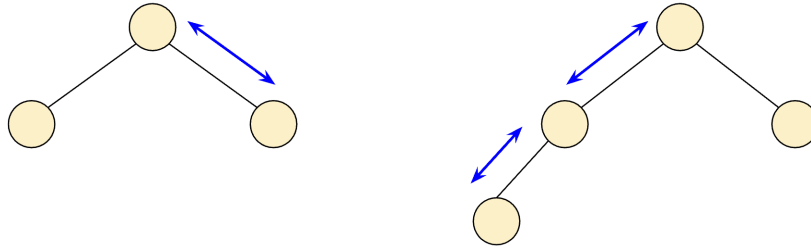
In the figure above, $y = Q(N)$ is plotted as a black curve, using points that are marked. The OOG of $Q(N)$ seems to be $O(N^2)$ or less, but it is definitely greater than $O(N)$ as there is an increasing gradient. Since this finding did not present a coherent indication of the OOG of $Q(N)$, I decided to utilise upper and lower bounds of $Q(N)$ to narrow down its OOG, demonstrated in the following sections.

4.2 Finding the upper bound

I will define the upper bound of $Q(N)$ as $A(N)$, where $O(A(N))$ is less than $O(N^2)$ but greater than or equal to $O(Q(N))$.

In figure 12, it is shown that each node that is inserted is not always swapped the same amount of times as other nodes that are inserted, even in a worse case scenario, since the maximum number of swaps that can happen is decided by the level at which the node was added at.

Figure 12: Difference in swaps as height increases



This is why $P(H) = \sum_{i=0}^H (2^i \times i)$ is difficult to calculate; there are variables in both the power and base of each term. If we over-estimate by assuming all nodes that are added are always swapped H amount of times, we would have a total of $N \times H$ operations.

$$\text{Recall (2), } H = \log_2(N + 1) - 1$$

$$\text{Actual maximum operations } Q(N) \leq N \times H$$

$$= N \times (\log_2(N + 1) - 1)$$

$$= N \log_2(N + 1) - N$$

$$< N \log_2(N + 1)$$

$$< N \log_2(N^2) \quad \text{for large values of } N$$

$$= 2N \log_2(N) = \text{upper bound of operations } A(N)$$

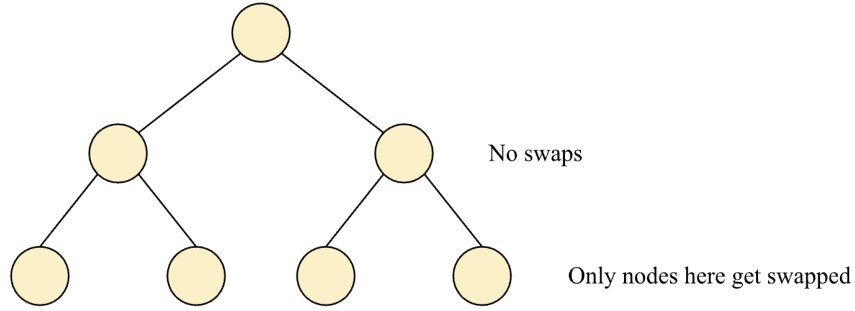
$$\in O(N \log(N))$$

Using inequalities, a loose upper bound, $A(N)$, was obtained with an OOG of $O(N \log(N))$. By definition of O -notation, the actual OOG of $Q(N)$ could either be $O(N \log(N))$, or less. To find it, I would have to obtain a lower bound for $Q(N)$ that is stricter with an OOG of $O(N \log(N))$, in order to form a tight bound for $Q(N)$.

4.3 Finding the lower bound

To obtain our lower bound, $B(N)$, I will underestimate the OPS function of Williams's method where it would be impossibly small as compared to the actual maximum operations, $Q(N)$. This is done by assuming that only the nodes on the last level of the binary heap would be allowed to swap.

Figure 13: Example scenario of the lower bound of $Q(N)$



As seen in figure 13, for all binary heaps, each node on the last level of the binary heap will be swapped until the root node, assuming a worse case scenario. This gives us our lower bound, $2^H \times H$.

$$\begin{aligned}
 \text{Actual maximum operations } Q(N) &\geq 2^H \times H \\
 &= 2^{\log_2(N+1)-1} \times (\log_2(N+1) - 1) \\
 &= \frac{2^{\log_2(N+1)}}{2} \times (\log_2(N+1) - 1) \\
 &= \frac{N+1}{2} \times (\log_2(N+1) - 1) \\
 &> \frac{N}{2} \times (\log_2(N) - 1) \\
 &= \frac{N}{2} \times (\log_2(N) - \log_2(2)) \\
 &= \frac{N}{2} \times \log_2\left(\frac{N}{2}\right)
 \end{aligned}$$

$$\begin{aligned}
&> \frac{N}{2} \times \log_2(\sqrt{N}) \quad \text{for large values of } N \\
&= \frac{N}{2} \times \log_2(N^{\frac{1}{2}}) \\
&= \frac{N}{4} \times \log_2(N) \\
&= \text{lower bound of operations } B(N)
\end{aligned}$$

Hence, we get $A(N) = 2N\log_2(N)$ as our loose upper bound and $B(N) = \frac{N}{4}\log_2(N)$ as our loose lower bound; I can now find the OOG of $Q(N)$.

4.4 Obtaining the OOG

Although $A(N)$ applies when $N \geq 3$ and $B(N)$ applies when $N \geq 8$, this is acceptable since the OOG is calculated with large values of N . Because $Q(N)$ is exclusively between $A(N)$ and $B(N)$, which are made up of $N\log_2(N)$ multiplied by some positive constant, $Q(N)$, the OPS function of Williams's method, would have an OOG of $O(N\log(N))$.

After the invention of Williams's method, computer scientist R. W. Floyd found a faster way of creating a binary heap, which has an OPS function with a lower OOG than Williams's method (Li et al. 58).

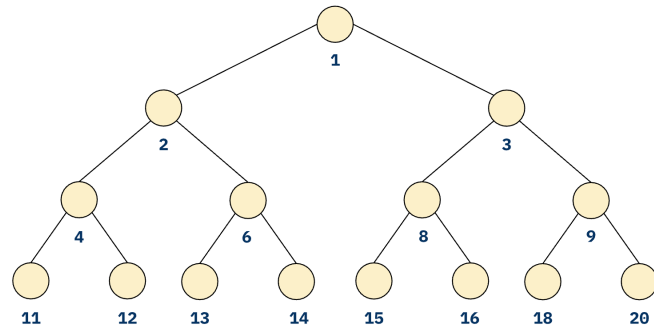
5 Floyd's binary heap creation method

The process of creating a binary heap with Floyd's method is shown through a step-by-step approach in figure 14.

Figure 14: Creation of a binary heap with Floyd's method

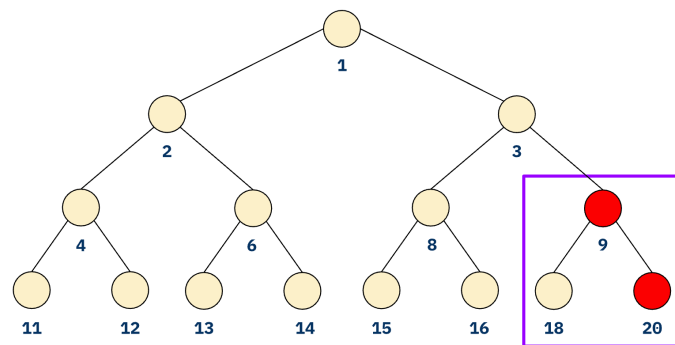
STEP 1

Elements to add:	1	2	3	4	6	8	9	11	12	13	14	15	16	18	20
------------------	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----



All 15 nodes are added to the binary heap, without any swaps done yet.

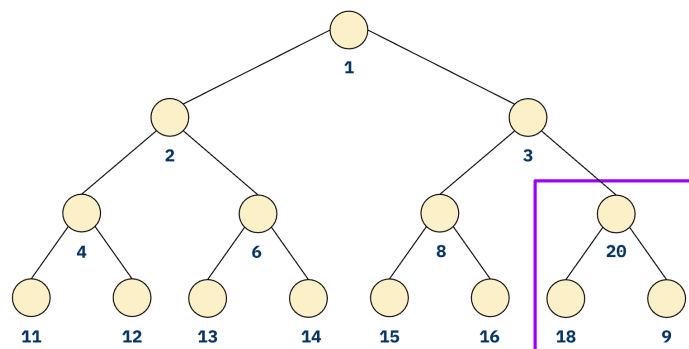
STEP 2



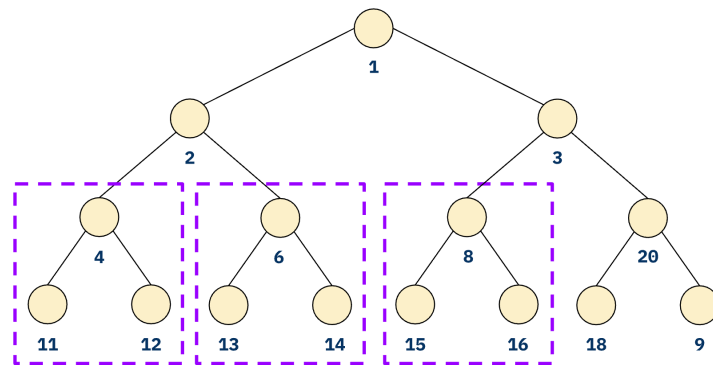
The nodes in the purple box are treated as an independent binary heap and its invalid nodes are marked in red.

$$9 < 18 < 20$$

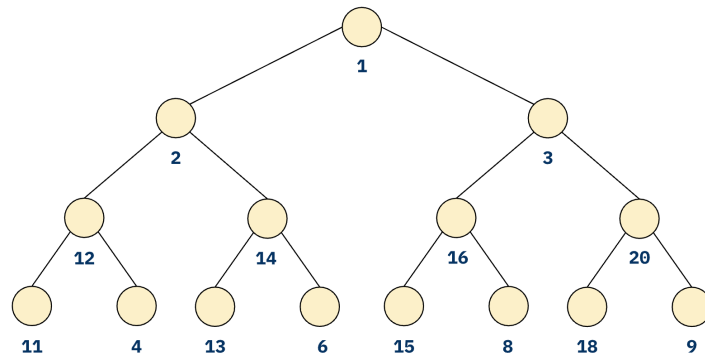
STEP 3



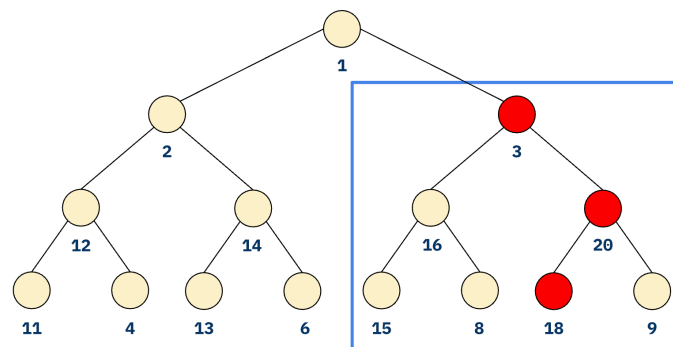
The invalid nodes in the purple box are swapped.

STEP 4

The process of steps 2 and 3 are repeated for the nodes in each dotted purple box.



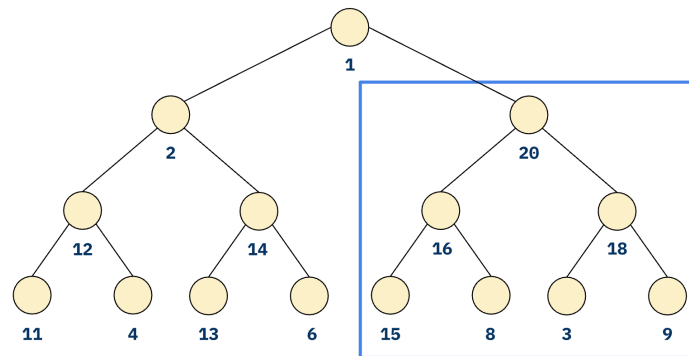
This is the result.

STEP 5

The nodes in the blue box are treated as an independent binary heap and its invalid nodes are marked in red.

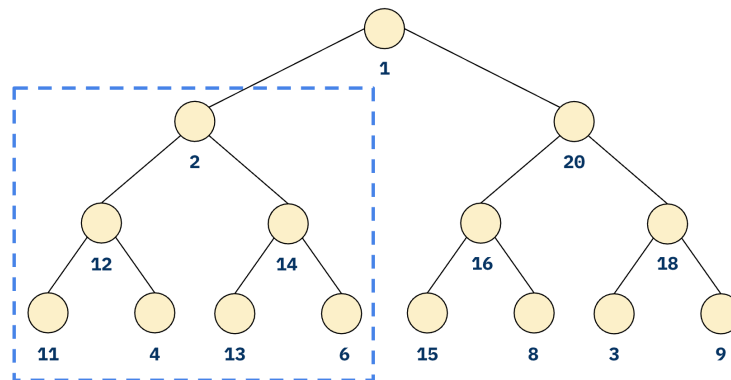
$$3 < 16 < 20 \text{ and } 3 < 9 < 18$$

STEP 6

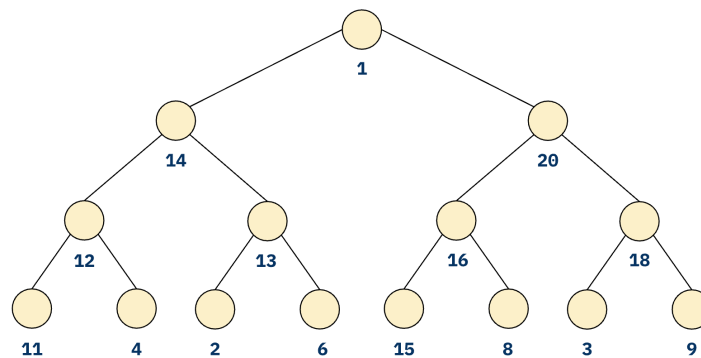


The invalid nodes in the blue box are swapped.

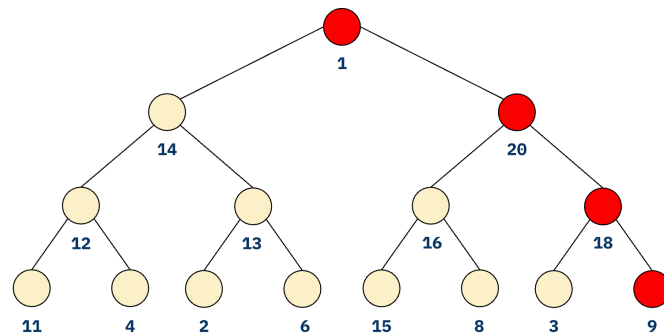
STEP 7



The process of steps 5 and 6 are repeated for the nodes in the dotted blue box.

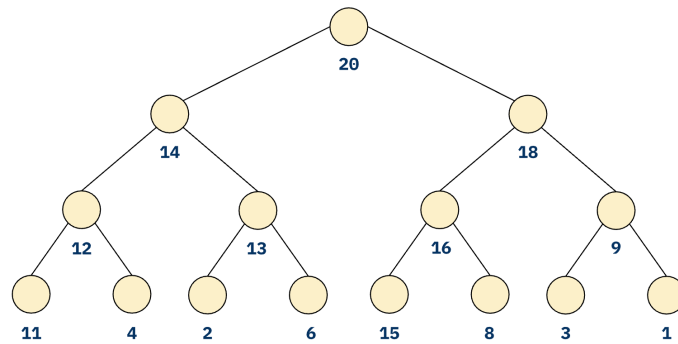


This is the result.

STEP 8

Now, in the whole binary heap, invalid nodes are marked out in red.

$1 < 14 < 20$, $1 < 16 < 18$ and $1 < 3 < 9$

STEP 9

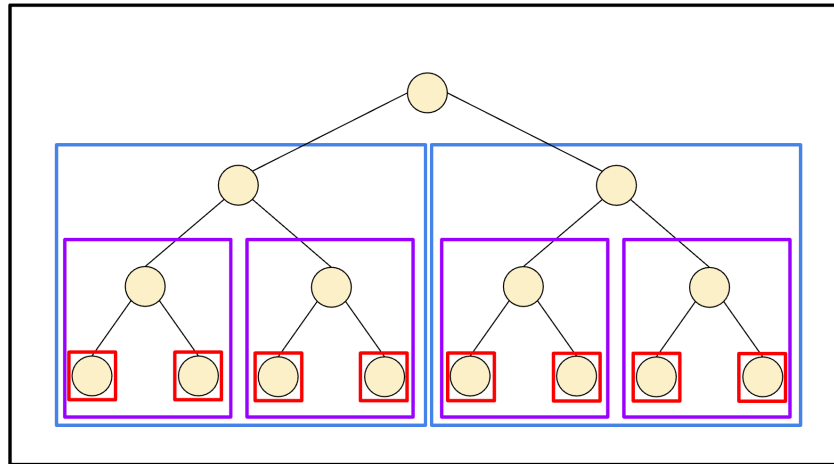
Invalid nodes are swapped. The binary heap is now valid.

In the figure above, notice that when a parent node has a lower priority than one or both its child nodes, the child node with the higher priority swaps with the parent node. This is to ensure that the priority of the new parent node will be greater than that of both its child nodes.

Floyd's method, as demonstrated in figure 14, conceptually breaks down an invalid binary heap into smaller parts. What is not shown in figure 14 is that each node at

the bottom level is first treated as an independent binary heap, indicated by the red boxes shown in figure 15.

Figure 15: Floyd's method breakdown



Since a binary heap containing one node is always sorted, no swaps occur within the red boxes. Next, the nodes in the purple boxes are rearranged, followed by the nodes in the blue boxes and finally with the entire binary heap. In each of the boxes that the original binary heap is split into, nodes are swapped so that they become a valid binary heap (Lee 13). Figures 14 and 15 both use an example of a binary heap with $N = 15$, but this process applies to binary heaps of all positive integer values of N .

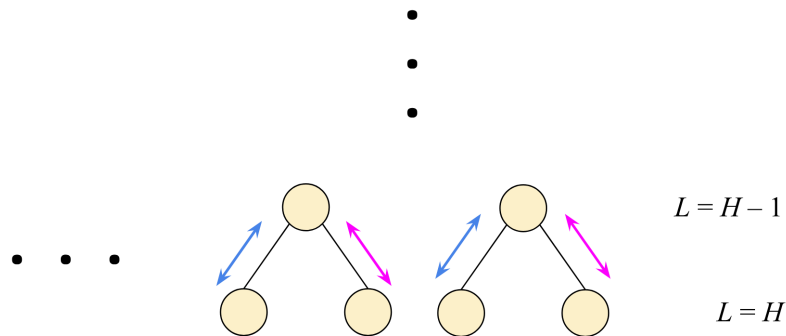
To investigate why Floyd's method is faster than Williams's method, I will need to obtain the OOG of the OPS function of Floyd's method so that it can be compared to that of Williams's method.

5.1 Obtaining the OPS function

We shall assume a worst case scenario where all elements that are being added to the binary heap would have a higher priority than the previous element, with the root node having the lowest priority before any swapping has occurred. In order to find the OOG of the OPS function of Floyd's method, I would need to obtain the relationship between N and the total number of operations, which is the number of swaps, needed for the creation of a binary heap.

Figure 16 shows the first 2 groups of nodes that are rearranged with a binary heap of N elements, with all parent nodes having a lower priority than both their child nodes.

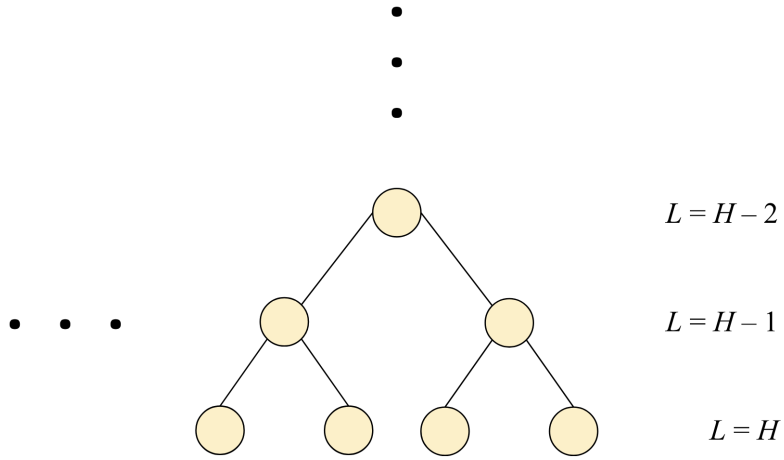
Figure 16: Swapping of nodes at level $H - 1$



The parent nodes at $L = H - 1$ are either swapped with its left child node or its right child node, depending on which of the two child nodes has the highest priority. This means that there is a maximum of 1 operation done for each node at $L = H - 1$.

Once all the nodes at $L = H - 1$ are swapped, Floyd's method would then swap invalid nodes at $L = H - 2$, depicted in Figure 17.

Figure 17: Swapping of nodes at level $H - 2$



In a worse case scenario, the maximum number of swaps for nodes on a certain level is equal to how many levels they are from the bottom of the binary heap as nodes can only be swapped with their own child nodes. Since nodes at $L = H - 2$ are 2 levels from level H , we can say that there is a maximum of 2 operations done for every node at $L = H - 2$. By representing the maximum number of operations per node at level L as M , I am able to find M in terms L and H :

$$L = H - M$$

$$L + M = H$$

$$M = H - L \tag{3}$$

Now, recalling the definition and use of L from section 3.1, the maximum number of operations of all nodes at level L is $2^L \times M$. By substituting M as in (3),

$$\text{Max no. of operations of all nodes at level } L = 2^L \times (H - L)$$

Hence, I can define function $F(H)$ as the total number of operations required by Floyd's method for a binary heap of height H , expressed as:

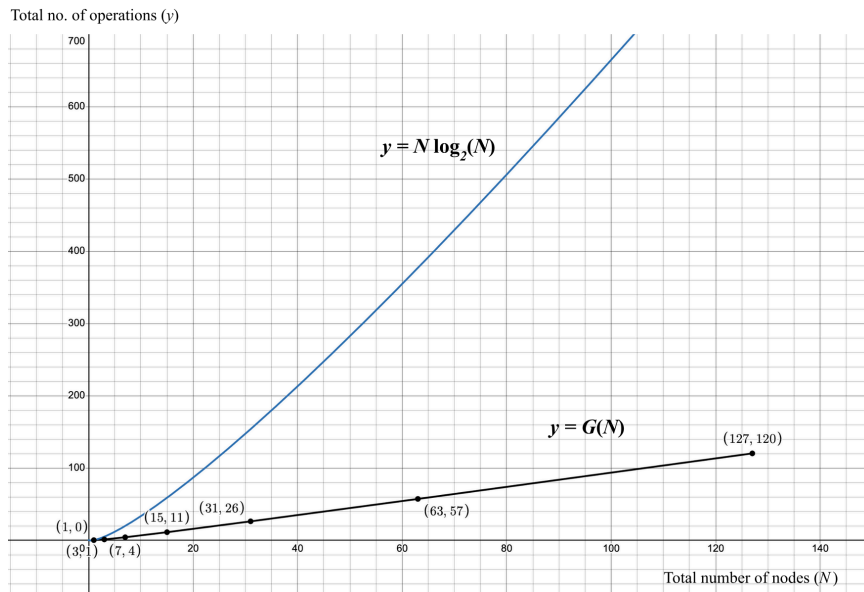
$$F(H) = \sum_{i=0}^H (2^i \times (H - i))$$

Recalling (1), I can rewrite $F(H)$ in terms of N where $G(N)$ is the OPS function of Floyd's method and $G(N) \equiv F(H)$:

$$G(N) = \sum_{i=0}^{\log_2(N+1)-1} (2^i \times (\log_2(N+1) - 1 - i))$$

By plotting the first few points of $G(N)$ in figure 18, we can see that the OOG of $G(N)$ is much lower than $N \log_2(N)$. In fact, it appears to have a linear OOG, with an R^2 value of 0.9994 when a linear regression line of $y = 0.958661N - 2.54134$ is applied to the points.

Figure 18: Plotting values of $N \log_2(N)$ and $G(N)$



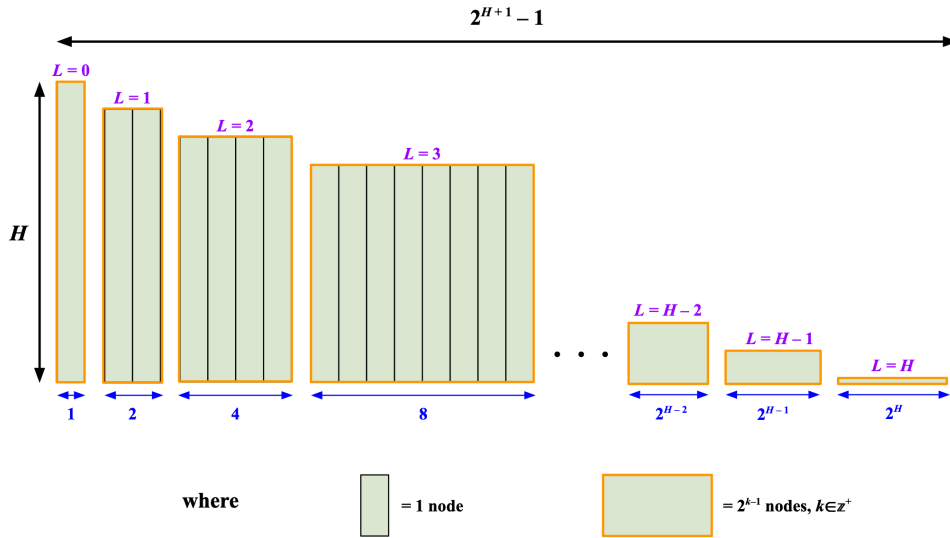
Unfortunately, concluding that $G(N)$ has a linear OOG would not be a strong claim as large values of N are not found on the empirical evidence and a regression model should not be used for extrapolation; the OOG of $G(N)$ for large values of N remains unknown to me.

However, I realised that we could utilise geometry to analyse $F(H) = \sum_{i=0}^H (2^i \times (H - i))$ further, demonstrated in the next section.

5.2 Use of geometry to transform the problem

As defined previously, $F(H)$ is the summation of the maximum number of operations of all nodes at level L . In figure 19, each term in the series of $F(H)$ is drawn as a rectangle where each rectangle's area corresponds to $2^L \times (H - L)$ at different values of L , indicated by the purple text.

Figure 19: Expressing terms in $F(H)$ using 2D geometry

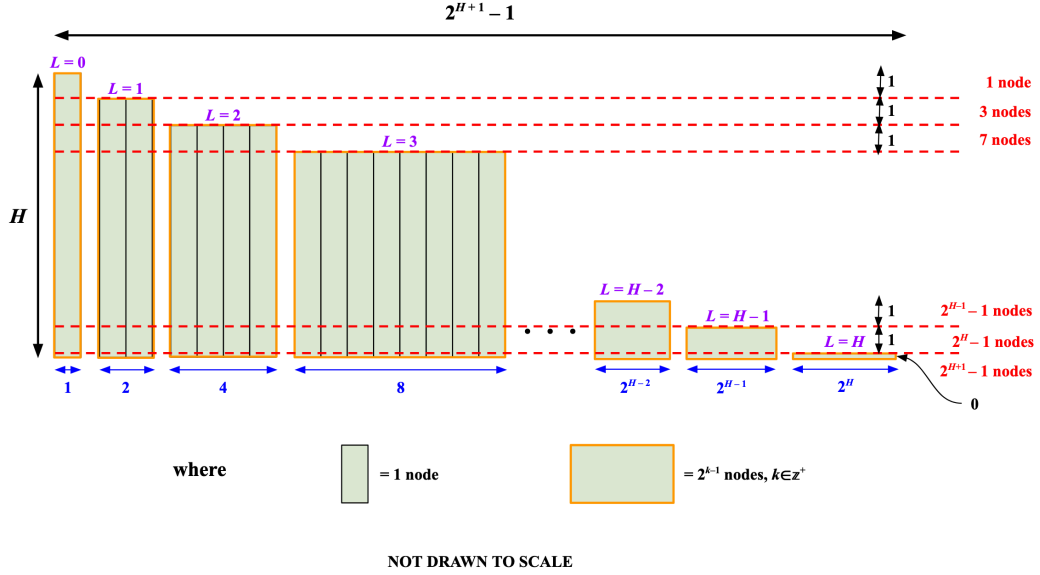


NOT DRAWN TO SCALE

The width of each rectangle is represented by the number of nodes at the current level, and their respective heights represented by the maximum number of swaps for each node in that level; the number of levels from the bottom of the binary heap. By laying the rectangles side by side on the horizontal axis, we can see that the sum of their widths is the total number of nodes, N , which can also be expressed as $2^{H+1} - 1$. $F(H)$ is therefore the total area of H rectangles, one rectangle for each term in the series. Note that the last term is technically not a rectangle as it would

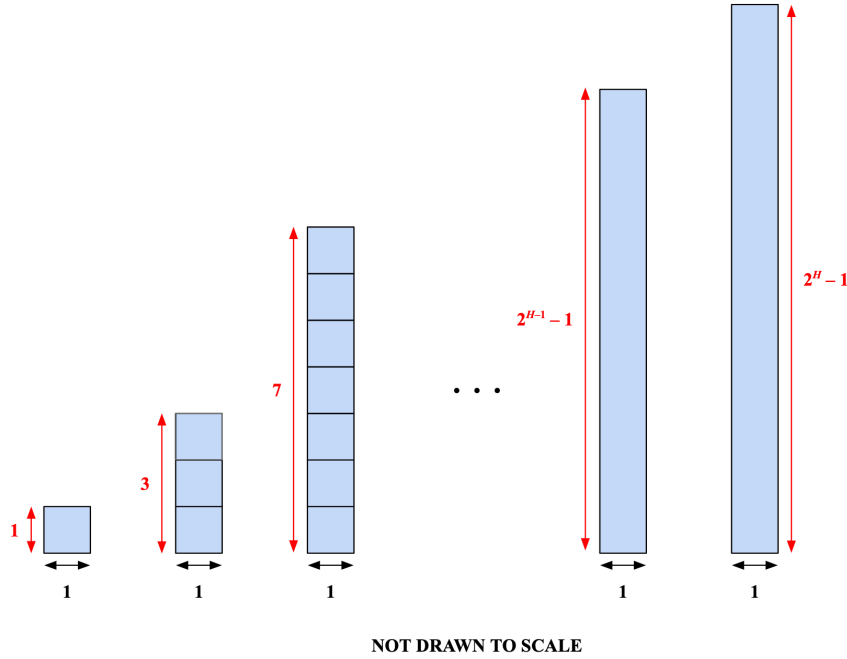
have a height of 0. My main goal would be to find the area of all nodes in figure 19 such that $F(H)$ can be expressed in a way that is easier to simplify compared to $\sum_{i=0}^H (2^i \times (H - i))$.

Figure 20: Splitting terms in geometric representation of $F(H)$



In figure 20, I cut the rectangles horizontally into parts of equal height, 1, since the distance between levels L and H decreases by 1 as L increases by 1. Each part is a rectangle of height 1, or width of 1 when it is rotated 90 degrees in any direction. Note that the nodes at level H would have a distance to the bottom of the binary heap of 0.

Figure 21: Representation of cut rectangles from figure 20



We can now see that the area of figure 21, which is the same as that of figure 20, can be calculated by applying a summation to $2^H - 1$:

$$\begin{aligned}
 F(H) &= \sum_{i=1}^H (2^i - 1) \\
 &= \sum_{i=1}^H (2^i) - H \times 1
 \end{aligned}$$

$\sum_{i=1}^H (2^i)$ is a geometric series with a common ratio of 2. Therefore, I can use the formula for the sum of a geometric series to simplify $F(H)$ and obtain its OOG in the following section.

5.3 Obtaining the OOG

According to the the International Baccalaureate, the formula to find the sum of n terms for a finite geometric series is given as: $S_n = \frac{u_1(r^n - 1)}{r - 1}$ where u_1 is the first

term of the series and r is the common ratio of the series. Using this, $F(H)$ and its equivalent function in terms of N , $G(N)$, can be simplified:

$$\begin{aligned}
 F(H) &= \sum_{i=1}^H (2^i) - H \\
 &= \frac{2(2^H - 1)}{2 - 1} - H \quad \because S_n = \frac{u_1(r^n - 1)}{r - 1} \\
 &= 2(2^H - 1) - H
 \end{aligned}$$

Recalling (2), $H = \log_2(N + 1) - 1$

$$\begin{aligned}
 G(N) &\equiv F(H) \\
 &= 2(2^{\log_2(N+1)-1} - 1) - (\log_2(N + 1) - 1) \\
 &= 2^{\log_2(N+1)-1+1} - 2 - \log_2(N + 1) + 1 \\
 &= 2^{\log_2(N+1)} - 1 - \log_2(N + 1) \\
 &= (N + 1) - 1 - \log_2(N + 1) \\
 &= N - \log_2(N + 1) \\
 \therefore \frac{N}{2} &< G(N) < N \text{ for large values of } N
 \end{aligned}$$

Since $G(N)$ is exclusively between $\frac{N}{2}$ and N for large values of N , it can be concluded that $G(N) \in O(N)$. Hence, $G(N)$ has been proven to have a linear OOG where $G(N)$ grows in a manner approximately similar to the product of k and N for large values of N with k as some small constant.

6 Conclusion

When creating a binary heap, Floyd's algorithm is faster than Williams's algorithm since the OOG of the OPS function of Williams's method is $O(N \log N)$ and the

OOG of the OPS function of Floyd's method is $O(N)$. This means that Floyd's method performs less operations for large values of N , because the number of operations required to create a binary heap grows slower as N grows, in comparison to Williams's method.

While the speed of Floyd's method has been widely proven, the proofs often require tertiary-level knowledge or calculus (Lee 17; Skiena 27-28). In this essay, I have **found a novel approach** involving the **transformation of the problem**, seen in figures 19 to 21, using simple sequences and geometry which allows the reason behind Floyd's faster method of creating binary heaps to be **more easily understood**.

7 Bibliography

- Cormen, Thomas H., et al. Introduction to Algorithms, Third Edition. 2009, pp. 43–60. archive.org/details/introduction-to-algorithms-third-edition-2009/page/43/mode/1up. Accessed 4 May 2025.
- Cornell University. Priority Queues and Heaps. 2020. www.cs.cornell.edu/courses/cs2112/2020fa/lectures/lecture.html?id=heaps. Accessed 18 Jan. 2025.
- Demaine, Erik, et al. Introduction to Algorithms: 6.006, Lecture 8: Binary Heaps. 2020. ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/40d4851e550507ca14dc778b9b2266cc/MIT6_006S20_lec8.pdf. Accessed 16 May 2025.
- Eldridge, Stephen. time complexity. 1 Sept. 2023. www.britannica.com/science/time-complexity. Accessed 18 May 2025.
- Huang, Shen. What is Big O Notation Explained: Space and Time Complexity. Jan. 2020. www.freecodecamp.org/news/big-o-notation-why-it-matters-and-why-it-doesnt-1674cfa8a23c/. Accessed 11 Jan. 2025.
- International Baccalaureate. Mathematics: analysis and approaches formula booklet. 2021, p. 3. www.fjuhsd.org/cms/lib/CA02000098/Centricity/Domain/233/Formula%20booklet%20SL%20and%20HL%20May%202021.pdf. Accessed 23 Mar. 2025.
- Lee, Michael. CSE 373: Floyd’s buildHeap algorithm; divide-and-conquer. 7 Feb. 2018, pp. 15–17. courses.cs.washington.edu/courses/cse373/18wi/files/slides/lecture-14-6up.pdf. Accessed 17 May 2025.

- Li, Haiming, et al. "Heap Sorting Based on Array Sorting". *Journal of Computer and Communications*, vol. 05, 30 Oct. 2017, pp. 57–59.
<https://doi.org/10.4236/jcc.2017.512006>.
- National University of Singapore Computing. Binary Heap (Priority Queue). Pp. 1, 3, 4. visualgo.net/en/heap?slide=1-1. Accessed 7 Jan. 2025.
- Schwarz, Keith. Guide to Big-O Notation. Nov. 2022. web.stanford.edu/class/archive/cs/cs106b/cs106b.1256/resources/bigo_guide.html. Accessed 4 May 2025.
- Sirois, Sophie. Understanding CPU Speed: What Makes a Good Processor for Your Computer. 2 Aug. 2024.
www.hp.com/us-en/shop/tech-takes/what-is-processor-speed. Accessed 13 May 2025.
- Skiena, Steven. Lecture 7: Heapsort / Priority Queues. Pp. 25–28.
www3.cs.stonybrook.edu/~skiena/373/oldlectures/lecture7.pdf. Accessed 7 Jan. 2025.
- Williams, J. W. J. Algorithm 232: Heapsort. *Communications of the ACM*, edited by G. E. Forsythe, vol. 7, 6, June 1964, pp. 347–49.
<https://doi.org/10.1145/512274.3734138>.